

LLM & Embedding API — Panduan Pengguna

Disusun oleh: Tim Kecerdasan Artifisial – Pusat Data dan Informasi

Versi: 1.0

Terakhir Diperbarui: Mei 2026

Penyedia: Badan Teknologi Informasi dan Intelijen Keuangan (BaTii) — Kementerian Keuangan RI

Ikhtisar

API ini menyediakan akses ke model-model *Large Language Model* (LLM) dan *Embedding* yang di-deploy pada server GPU NVIDIA H200 milik Kementerian Keuangan. API ini menggunakan format yang **sepenuhnya kompatibel dengan OpenAI**, sehingga *library* dan *framework* yang sudah mendukung OpenAI API (seperti **openai** Python SDK, LangChain, LlamaIndex) dapat langsung digunakan tanpa modifikasi kode yang signifikan.

Endpoint & Autentikasi

Base URL

Jalur	Base URL	Streaming	Keterangan
WebMethods (Eksternal)	`http://10.216.130.138:5555/llm/v1`	 Tidak didukung	Untuk integrasi API, skrip, *backend services*
Direct (Internal/Streaming)	`http://10.216.221.100/llm/v1`	 Didukung	Untuk aplikasi chat, Open WebUI, *real-time output*

Autentikasi

Setiap permintaan harus menyertakan **API Key** (Virtual Key) yang diterbitkan oleh tim Admin AI Pusdatin melalui header **Authorization**:

```
Authorization: Bearer <API_KEY_ANDA>
```

Hubungi tim AI Pusdatin untuk mendapatkan API Key Anda.

Model yang Tersedia

Model Chat (LLM)

Model ID	Mode	Cocok Untuk	Kecepatan
`qwen3.5-reasoning`	Penalaran (*Reasoning*)	Analisis kompleks, matematika, *coding*, perencanaan	Lebih lambat (berpikir dulu)
`qwen3.5-fast`	Langsung (*Direct*)	Tanya jawab sederhana, ringkasan, terjemahan	Cepat
`qwen3.6-reasoning`	Penalaran (*Reasoning*)	Analisis kompleks, matematika, *coding*, perencanaan	Lebih lambat (berpikir dulu)
`qwen3.6-fast`	Langsung (*Direct*)	Tanya jawab sederhana, ringkasan, terjemahan	Cepat

Kapan menggunakan model yang mana:

- **Model Reasoning** (*-reasoning*): Model akan memikirkan permasalahan secara bertahap sebelum menjawab. Gunakan untuk tugas yang memerlukan logika, analisis, atau penalaran multi-langkah.
- **Model Fast** (*-fast*): Model langsung menjawab tanpa proses berpikir. Gunakan untuk pertanyaan langsung, *chat*, dan tugas sederhana.

Model Embedding

Model ID	Dimensi	Keterangan
`qwen-embedding`	4096	Mengubah teks menjadi representasi vektor numerik untuk pencarian semantik

Endpoint API

Metode	*Path*	Deskripsi
`GET`	`/v1/models`	Menampilkan daftar model yang tersedia
`POST`	`/v1/chat/completions`	Inferensi chat (disarankan)
`POST`	`/v1/completions`	Inferensi *text completion*
`POST`	`/v1/embeddings`	Menghasilkan vektor *embedding* dari teks

1. Menampilkan Daftar Model

curl

```
curl http://10.216.130.138:5555/llm/v1/models \
-H "Authorization: Bearer <API_KEY_ANDA>"
```

Python

```
import requests

response = requests.get(
    "http://10.216.130.138:5555/llm/v1/models",
    headers={"Authorization": "Bearer <API_KEY_ANDA>"}
)

for model in response.json()["data"]:
    print(f" {model['id']} - {model.get('description', '')}")
```

Contoh Respons

```
{
  "object": "list",
  "data": [
    {"id": "qwen3.5-reasoning", "object": "model", "description": "Qwen3.5 27B - Reasoning mode"},
    {"id": "qwen3.5-fast", "object": "model", "description": "Qwen3.5 27B - Fast mode"},
    {"id": "qwen3.6-reasoning", "object": "model", "description": "Qwen3.6 35B FP8 MoE - Reasoning mode"},
    {"id": "qwen3.6-fast", "object": "model", "description": "Qwen3.6 35B FP8 MoE - Fast mode"},
    {"id": "qwen-embedding", "object": "model", "description": "Qwen3 Embedding 8B"}
  ]
}
```

2. Chat Completion — Model Reasoning

Model *reasoning* akan menyertakan blok *thinking* yang berisi proses berpikir sebelum memberikan jawaban akhir.

curl

```
curl http://10.216.130.138:5555/llm/v1/chat/completions \
-H "Authorization: Bearer <API_KEY_ANDA>" \
-H "Content-Type: application/json" \
-d '{
  "model": "qwen3.5-reasoning",
  "messages": [
    {"role": "system", "content": "Anda adalah asisten profesional Kementerian Keuangan."},
    {"role": "user", "content": "Jelaskan secara ringkas konsep Retrieval-Augmented Generation (RAG)."}
  ],
  "max_tokens": 500,
  "temperature": 0.7
}'
```

Python

```
import requests
```

```

response = requests.post(
    "http://10.216.130.138:5555/llm/v1/chat/completions",
    headers={
        "Authorization": "Bearer <API_KEY_ANDA>",
        "Content-Type": "application/json"
    },
    json={
        "model": "qwen3.5-reasoning",
        "messages": [
            {"role": "system", "content": "Anda adalah asisten profesional Kementerian Keuangan."},
            {"role": "user", "content": "Jelaskan secara ringkas konsep Retrieval-Augmented Generation (RAG)."}
        ],
        "max_tokens": 500,
        "temperature": 0.7
    }
)

print(response.json()["choices"][0]["message"]["content"])

```

Contoh Respons

```

{
  "id": "chatcmpl-abc123",
  "object": "chat.completion",
  "choices": [
    {
      "index": 0,
      "message": {
        "role": "assistant",
        "content": "RAG (Retrieval-Augmented Generation) adalah teknik yang menggabungkan pencarian dokumen dengan generasi teks oleh LLM. Prosesnya meliputi:\n1. Mengubah pertanyaan pengguna menjadi vektor embedding\n2. Mencari dokumen relevan di database vektor\n3. Menyertakan dokumen tersebut sebagai konteks untuk LLM\n4. LLM menghasilkan jawaban berdasarkan konteks tersebut"
      },
      "finish_reason": "stop"
    }
  ],
  "usage": {
    "prompt_tokens": 40,
    "completion_tokens": 180,
    "total_tokens": 220
  }
}

```

Catatan: Model reasoning menyertakan blok *thinking* yang berisi proses berpikir model. Blok ini dapat diabaikan atau disembunyikan dalam tampilan aplikasi Anda.

3. Chat Completion — Model Fast (Tanpa Proses Berpikir)

curl

```
curl http://10.216.130.138:5555/llm/v1/chat/completions \
-H "Authorization: Bearer <API_KEY_ANDA>" \
-H "Content-Type: application/json" \
-d '{
  "model": "qwen3.5-fast",
  "messages": [
    {"role": "user", "content": "Apa ibukota Indonesia?"}
  ],
  "max_tokens": 100
}'
```

Python

```
import requests

response = requests.post(
    "http://10.216.130.138:5555/llm/v1/chat/completions",
    headers={
        "Authorization": "Bearer <API_KEY_ANDA>",
        "Content-Type": "application/json"
    },
    json={
        "model": "qwen3.5-fast",
        "messages": [
            {"role": "user", "content": "Apa ibukota Indonesia?"}
        ],
        "max_tokens": 100
    }
)

print(response.json()["choices"][0]["message"]["content"])
```

Contoh Respons

```
{
  "choices": [
    {
      "message": {
        "role": "assistant",
        "content": "Ibukota Indonesia adalah Jakarta."
      },
      "finish_reason": "stop"
    }
  ],
  "usage": {
    "prompt_tokens": 12,
    "completion_tokens": 8,
    "total_tokens": 20
  }
}
```

Model *fast* memberikan jawaban langsung tanpa blok *thinking*, sehingga lebih hemat token dan lebih cepat.

4. Percakapan Multi-Giliran (*Multi-Turn*)

curl

```
curl http://10.216.130.138:5555/llm/v1/chat/completions \
-H "Authorization: Bearer <API_KEY_ANDA>" \
-H "Content-Type: application/json" \
-d '{
  "model": "qwen3.6-fast",
  "messages": [
    {"role": "system", "content": "Anda adalah asisten keuangan profesional."},
    {"role": "user", "content": "Apa itu PDB?"},
    {"role": "assistant", "content": "PDB adalah Produk Domestik Bruto, yaitu ukuran
total nilai barang dan jasa yang diproduksi dalam suatu negara pada periode tertentu."},
    {"role": "user", "content": "Bagaimana cara menghitungnya?"}
  ],
  "max_tokens": 300
}'
```

Python

```
import requests

messages = [
    {"role": "system", "content": "Anda adalah asisten keuangan profesional."},
    {"role": "user", "content": "Apa itu PDB?"},
    {"role": "assistant", "content": "PDB adalah Produk Domestik Bruto, yaitu ukuran total
nilai barang dan jasa yang diproduksi dalam suatu negara pada periode tertentu."},
    {"role": "user", "content": "Bagaimana cara menghitungnya?"}
]

response = requests.post(
    "http://10.216.130.138:5555/llm/v1/chat/completions",
    headers={
        "Authorization": "Bearer <API_KEY_ANDA>",
        "Content-Type": "application/json"
    },
    json={
        "model": "qwen3.6-fast",
        "messages": messages,
        "max_tokens": 300
    }
)

print(response.json()["choices"][0]["message"]["content"])
```

5. Embedding (Mengubah Teks Menjadi Vektor)

Model *embedding* mengubah teks menjadi representasi vektor numerik berdimensi tinggi. Vektor ini memuat makna semantik dari teks, sehingga komputer dapat "memahami" konteks dan hubungan antar kalimat. Embedding merupakan fondasi penting untuk RAG (*Retrieval-Augmented Generation*).

curl

```
curl http://10.216.130.138:5555/llm/v1/embeddings \
-H "Authorization: Bearer <API_KEY_ANDA>" \
-H "Content-Type: application/json" \
-d '{
  "model": "qwen-embedding",
  "input": "Kementerian Keuangan Republik Indonesia"
}'
```

Python

```
import requests

response = requests.post(
    "http://10.216.130.138:5555/llm/v1/embeddings",
    headers={
        "Authorization": "Bearer <API_KEY_ANDA>",
        "Content-Type": "application/json"
    },
    json={
        "model": "qwen-embedding",
        "input": "Kementerian Keuangan Republik Indonesia"
    }
)

embedding_vector = response.json()["data"][0]["embedding"]
print(f"Dimensi vektor: {len(embedding_vector)}")
print(f"5 nilai pertama: {embedding_vector[:5]}")
```

Embedding Banyak Teks Sekaligus (*Batch*)

```
import requests

response = requests.post(
    "http://10.216.130.138:5555/llm/v1/embeddings",
    headers={
        "Authorization": "Bearer <API_KEY_ANDA>",
        "Content-Type": "application/json"
    },
    json={
        "model": "qwen-embedding",
        "input": [
            "Peraturan Menteri Keuangan tentang Tata Kelola Data",
            "Laporan Keuangan Pemerintah Pusat Tahun 2025",
            "Kebijakan fiskal untuk mendukung pertumbuhan ekonomi"
        ]
    }
)
```

```
for item in response.json()["data"]:
    print(f"Index {item['index']}: dimensi = {len(item['embedding'])}")
```

Contoh Respons Embedding

```
{
  "object": "list",
  "data": [
    {
      "object": "embedding",
      "index": 0,
      "embedding": [0.01234, -0.09876, 0.04567, "..."]
    }
  ],
  "model": "qwen-embedding",
  "usage": {
    "prompt_tokens": 8,
    "total_tokens": 8
  }
}
```

6. Streaming (*Khusus Jalur Direct*)

Streaming memungkinkan Anda menerima respons dari model secara *real-time*, token per token, tanpa harus menunggu seluruh jawaban selesai dihasilkan. Ini sangat berguna untuk aplikasi *chat* yang membutuhkan umpan balik instan kepada pengguna.

⚠ **PENTING:** Streaming **hanya tersedia pada jalur Direct** (<http://10.216.221.100/llm/v1>). Jalur WebMethods (<http://10.216.130.138:5555/llm/v1>) **tidak mendukung streaming** karena keterbatasan arsitektur *thread-per-request* pada WebMethods.

Cara Kerja Streaming

Ketika parameter **"stream": true** disertakan dalam permintaan, server akan mengirimkan respons dalam format **Server-Sent Events (SSE)**. Setiap potongan data (*chunk*) dikirim secara bertahap, diawali dengan prefiks **data:** , dan diakhiri dengan **data: [DONE]** ketika seluruh respons selesai.

```
data: {"id":"chatcmpl-xxx","choices":[{"delta":{"content":"Halo"}}]}
data: {"id":"chatcmpl-xxx","choices":[{"delta":{"content":"!"}}]}
data: {"id":"chatcmpl-xxx","choices":[{"delta":{"content":" Saya"}}]}
data: {"id":"chatcmpl-xxx","choices":[{"delta":{"content":" adalah"}}]}
...
data: [DONE]
```

Streaming dengan curl

Gunakan flag **-N** agar curl tidak melakukan *buffering* pada output, sehingga token langsung ditampilkan saat diterima:

```
curl -N http://10.216.221.100/llm/v1/chat/completions \
-H "Authorization: Bearer <API_KEY_ANDA>" \
-H "Content-Type: application/json" \
```

```
-d '{
  "model": "qwen3.5-fast",
  "messages": [
    {"role": "user", "content": "Tulis puisi pendek tentang data science dalam Bahasa Indonesia."}
  ],
  "max_tokens": 200,
  "stream": true
}'
```

Streaming dengan Python (Non-SDK, menggunakan `requests`)

Gunakan parameter `stream=True` pada method `requests.post()`, lalu iterasi setiap baris dari respons secara *real-time*:

```
import requests
import json

response = requests.post(
    "http://10.216.221.100/llm/v1/chat/completions",
    headers={
        "Authorization": "Bearer <API_KEY_ANDA>",
        "Content-Type": "application/json"
    },
    json={
        "model": "qwen3.5-fast",
        "messages": [
            {"role": "user", "content": "Tulis puisi pendek tentang data science dalam Bahasa Indonesia."}
        ],
        "max_tokens": 200,
        "stream": True
    },
    stream=True # Aktifkan streaming pada level HTTP
)

# Iterasi setiap baris dari respons SSE
for line in response.iter_lines():
    if line:
        decoded_line = line.decode("utf-8")

        # Lewati baris kosong atau komentar
        if not decoded_line.startswith("data: "):
            continue

        # Hapus prefiks "data: "
        data_str = decoded_line[len("data: "):]

        # Cek apakah stream sudah selesai
        if data_str.strip() == "[DONE]":
            print() # Baris baru di akhir
            break

        # Parse JSON dan tampilkan token
        try:
            chunk = json.loads(data_str)
```

```
        content = chunk["choices"][0]["delta"].get("content", "")
        if content:
            print(content, end="", flush=True)
    except json.JSONDecodeError:
        continue
```

Streaming dengan OpenAI Python SDK

Untuk pengguna yang lebih memilih menggunakan SDK resmi OpenAI, berikut contohnya. Pastikan menggunakan **jalur Direct**, bukan WebMethods:

```
from openai import OpenAI

client = OpenAI(
    base_url="http://10.216.221.100/llm/v1",
    api_key="<API_KEY_ANDA>"
)

stream = client.chat.completions.create(
    model="qwen3.6-fast",
    messages=[
        {"role": "user", "content": "Tulis puisi pendek tentang data science."}
    ],
    max_tokens=200,
    stream=True
)

for chunk in stream:
    if chunk.choices[0].delta.content:
        print(chunk.choices[0].delta.content, end="", flush=True)
print()
```

7. Menggunakan OpenAI Python SDK (Non-Streaming)

API ini sepenuhnya kompatibel dengan *library* **openai** untuk Python.

Instalasi

```
pip install openai
```

Penggunaan Dasar

```
from openai import OpenAI

client = OpenAI(
    base_url="http://10.216.130.138:5555/llm/v1",
    api_key="<API_KEY_ANDA>"
)

# Daftar model
models = client.models.list()
for model in models.data:
    print(f" {model.id}")
```

```
# Chat completion (reasoning)
response = client.chat.completions.create(
    model="qwen3.5-reasoning",
    messages=[
        {"role": "user", "content": "Buatkan fungsi Python untuk menghitung faktorial."}
    ],
    max_tokens=500
)
print(response.choices[0].message.content)
```

Mode Fast

```
response = client.chat.completions.create(
    model="qwen3.5-fast",
    messages=[
        {"role": "user", "content": "Berapa 15% dari 200?"}
    ],
    max_tokens=50
)
print(response.choices[0].message.content)
# Output: "15% dari 200 adalah 30."
```

Embedding via SDK

```
response = client.embeddings.create(
    model="qwen-embedding",
    input="Kementerian Keuangan Republik Indonesia"
)
print(f"Dimensi: {len(response.data[0].embedding)}")
```

Parameter Permintaan

Parameter Wajib

Parameter	Tipe	Deskripsi
`model`	string	ID model yang digunakan (lihat tabel model di atas)
`messages`	array	Daftar pesan percakapan (untuk chat)
`input`	string/array	Teks yang akan di-*embed* (untuk embedding)

Parameter Opsional (Chat)

Parameter	Tipe	Default	Rentang	Deskripsi
`max_tokens`	integer	1024	1 — 32768	Jumlah maksimum token yang dihasilkan
`temperature`	float	0.7	0.0 — 2.0	Mengontrol keacakan. Rendah = lebih fokus, tinggi = lebih kreatif
`top_p`	float	1.0	0.0 — 1.0	*Nucleus sampling*

`frequency_penalty`	float	0.0	-2.0 — 2.0	Mengurangi pengulangan kata
`presence_penalty`	float	0.0	-2.0 — 2.0	Mendorong variasi topik
`stop`	string/array	null	—	Menghentikan generasi saat string ini ditemukan
`stream`	boolean	false	—	Mengaktifkan <i>*streaming*</i> (**hanya jalur Direct**)
`seed`	integer	null	—	Untuk hasil yang dapat direproduksi

Format Pesan

```
{
  "messages": [
    {"role": "system", "content": "Instruksi untuk perilaku model (opsional)"},
    {"role": "user", "content": "Pertanyaan atau input pengguna"},
    {"role": "assistant", "content": "Respons model sebelumnya (untuk multi-giliran)"},
    {"role": "user", "content": "Pertanyaan lanjutan"}
  ]
}
```

Role	Fungsi	Wajib?
`system`	Mengatur perilaku dan kepribadian model	Opsional
`user`	Input/pertanyaan dari pengguna	Minimal satu
`assistant`	Respons model sebelumnya (untuk konteks percakapan)	Opsional

Panduan Penggunaan Parameter

Panduan `temperature`

Nilai	Karakteristik	Cocok Untuk
`0.0`	Jawaban sama setiap kali	Fakta, matematika, kode
`0.3`	Sangat fokus	Analisis, ringkasan, terjemahan
`0.7`	Seimbang (default)	Chat umum, penjelasan
`1.0+`	Sangat kreatif	Brainstorming, penulisan kreatif

Panduan `max\_tokens`

Nilai	Perkiraan Hasil
`50`	Jawaban satu kalimat
`100`	Paragraf pendek
`300`	Paragraf detail
`500`	Setengah halaman
`1000`	Satu halaman penuh
`2000+`	Respons panjang / generasi kode

Rekomendasi Pengaturan per Tugas

Tugas	Model	Temperature	Max Tokens
Tanya jawab sederhana	`*-fast`	0.3	100
Ringkasan dokumen	`*-fast`	0.3	500
Terjemahan	`*-fast`	0.0	300
Analisis mendalam	`*-reasoning`	0.5	1000
Generasi kode	`*-reasoning`	0.2	2000
Penulisan kreatif	`*-reasoning`	0.9	1000
Ekstraksi data	`*-fast`	0.0	500
Soal matematika	`*-reasoning`	0.1	1000
Chatbot umum	`*-fast`	0.7	500
Embedding dokumen	`qwen-embedding`	—	—

Contoh Prompt Sistem

```
# Asisten formal pemerintah
{"role": "system", "content": "Anda adalah asisten profesional Kementerian Keuangan RI. Selalu berikan jawaban dalam Bahasa Indonesia yang formal, jelas, dan terstruktur sesuai dokumen resmi pemerintah."}

# Ahli kode
{"role": "system", "content": "Anda adalah pengembang Python senior. Selalu sertakan type hints dan docstring dalam kode Anda."}

# Analis data
{"role": "system", "content": "Anda adalah analis data. Ketika diberikan data, berikan insight statistik dan rekomendasikan visualisasi yang sesuai."}

# Respons singkat
{"role": "system", "content": "Jawab dalam 2-3 kalimat maksimal. Langsung dan ringkas."}
```

Format Respons

Respons Non-Streaming (Standar)

```
{
  "id": "chatcmpl-unique-id",
  "object": "chat.completion",
  "model": "nama-model",
  "choices": [
    {
      "index": 0,
      "message": {
        "role": "assistant",
        "content": "Teks respons model di sini."
      },
      "finish_reason": "stop"
    }
  ]
}
```

```

    ],
    "usage": {
      "prompt_tokens": 25,
      "completion_tokens": 50,
      "total_tokens": 75
    }
  }
}

```

Respons Streaming (Per Chunk)

Setiap chunk dikirim sebagai Server-Sent Event (SSE):

```

{
  "id": "chatcmpl-unique-id",
  "object": "chat.completion.chunk",
  "choices": [
    {
      "index": 0,
      "delta": {
        "content": "satu-atau-beberapa-kata"
      },
      "finish_reason": null
    }
  ]
}

```

Chunk terakhir menandakan selesai:

```
data: [DONE]
```

Penjelasan Field Respons

Field	Deskripsi
`choices[0].message.content`	Teks respons dari model (non-streaming)
`choices[0].delta.content`	Potongan teks per token (streaming)
`choices[0].finish_reason`	``stop`` = selesai, ``length`` = mencapai batas `max_tokens`
`usage.prompt_tokens`	Token yang digunakan oleh input Anda
`usage.completion_tokens`	Token yang dihasilkan oleh model
`usage.total_tokens`	Total token yang dikonsumsi

Catatan Penting tentang Streaming

Jalur	Streaming	Penggunaan
WebMethods (`10.216.130.138:5555/llm/v1`)	❌ Tidak didukung	Integrasi API, skrip, *backend services*
Direct (`10.216.221.100/llm/v1`)	✅ Didukung	Aplikasi chat, *real-time output*

⚠️ **Jangan gunakan ``stream``: true` pada jalur WebMethods karena akan menyebabkan *timeout* atau respons kosong.**

Respons Error

Kode Status	Arti	Contoh
`401`	API key tidak valid atau tidak disertakan	`{"error": "Unauthorized"}`
`404`	Nama model tidak ditemukan	`{"detail": "Model 'nama-salah' not found. Available: [...]}"`
`429`	Batas *rate limit* terlampaui	`{"error": "Rate limit exceeded"}`
`500`	Kesalahan server	Hubungi tim AI Pusdatin (BaTii)

Dukungan

Kebutuhan	Kontak
Permintaan API Key baru	Tim AI Pusdatin (BaTii)
API tidak merespons	Tim AI Pusdatin (BaTii)
Panduan pemilihan model	Lihat dokumentasi ini
Pertanyaan penggunaan	Lihat dokumentasi ini